

Atelier 3 — Le transfert du site (FTP / FTPS)

Fiche de révision fusionnée — BTS SIO

Contents

Objectif de l'atelier	1
1. Serveurs FTP existants sur le marché	1
2. Mise en place du serveur FTP (phase 1 — TypSoft)	2
3. Client FTP et tests de transfert	2
4. Documentation utilisateur / technicien	2
5. Analyse de sécurité avec Wireshark — la faille du FTP classique	3
6. Migration vers le FTPS (sécurisation)	3
Conclusion type	3

Objectif de l'atelier

Comprendre comment transférer des fichiers entre un poste de développement et un serveur web (mise en ligne / mise à jour du site GSB), puis identifier et corriger les failles de sécurité du protocole FTP classique.

1. Serveurs FTP existants sur le marché

Serveur	OS	Points forts
FileZilla Server	Windows/Linux	Gratuit, open-source, GUI, supporte FTP et FTPS — très utilisé en milieu scolaire/PME
TypSoft FTP Server	Windows	Gratuit, léger, simple à mettre en route — idéal pour un premier test / des TP
vsftpd	Linux	Open-source, réputé pour sa sécurité et ses performances — usage professionnel
ProFTPD	Linux/Unix	Open-source, configuration avancée, bonne gestion des droits
Serv-U	Windows/Linux	Payant, supporte FTP/FTPS/SFTP — entreprises

Serveur	OS	Points forts
IIS FTP	Windows Server	Intégré à Windows Server — environnements Microsoft

2. Mise en place du serveur FTP (phase 1 — TypSoft)

1. Installer **TypSoft FTP Server** (ftpserv_1.1.exe), l'extraire dans un dossier dédié (ex. C:\FTPServer\Typsoft), le lancer **en tant qu'administrateur** (obligatoire : sinon impossible d'écouter sur le port 21, réservé).
2. Répertoire racine FTP défini, ex. C:\FTPRoot / C:\FTP. Port par défaut : **21**.
3. Créer les comptes utilisateurs avec des droits différenciés selon le rôle métier :

Utilisateur	Répertoire	Droits
développeur	... \dev	Lecture, Écriture, Listing (pas de suppression → évite les accidents en prod)
technicien	... \tech	Lecture, Écriture, Suppression, Listing
admin	C:\FTP	Tous droits (Full)

3. Client FTP et tests de transfert

- **Client utilisé : FileZilla Client** (gratuit, open-source, console de logs en temps réel).
- Connexion : IP du serveur (ex. 127.0.0.1), identifiant, mot de passe.
- **Test en mode graphique** : glisser-déposer un fichier (ex. index.html) vers le panneau serveur → transfert instantané, fichier bien reçu.
- **Test en ligne de commande (CMD)** : ftp 127.0.0.1, saisie login/mot de passe (masqué), puis put monfichier.html pour envoyer un fichier, get, dir, ls pour explorer/récupérer — utile pour préparer l'automatisation par script.
- **Vérification finale** : le site déposé s'affiche correctement via XAMPP/navigateur (http://localhost ou l'IP du serveur), testé sur Chrome et Firefox, y compris depuis un poste externe à la VM.

Point de vigilance rencontré : erreur 425 "Can't open data connection" en mode actif → passer en **mode passif** dans FileZilla (classique en présence d'un pare-feu entre client et serveur).

4. Documentation utilisateur / technicien

Deux documents distincts sont attendus car les publics diffèrent :

- **Guide développeur** : comment déposer son code sur le serveur FTP, dans quel répertoire, précautions pour ne pas écraser une version en production.
- **Guide technicien** : installation/configuration du serveur FTP, gestion des droits (NTFS/FTP) sur les dossiers, lecture des logs de connexion pour surveiller les accès.

Schéma de flux : *poste développeur* → *port 21* → *pare-feu GSB* → *serveur web (hébergeant le site)*.

5. Analyse de sécurité avec Wireshark — la faille du FTP classique

1. Lancer **Wireshark**, appliquer le filtre ftp pendant une session de connexion.
2. **Constat** : le nom d'utilisateur et le mot de passe apparaissent **en clair** dans la capture (commandes USER et PASS non chiffrées) — n'importe qui sur le même réseau peut les intercepter en quelques secondes.
3. **Conclusion** : le FTP standard, c'est comme envoyer une carte postale — tout le monde peut lire ce qui est écrit dessus. Inutilisable en production pour un site professionnel comme GSB.

6. Migration vers le FTPS (sécurisation)

Le **FTPS** = FTP encapsulé dans une couche **SSL/TLS** (comme HTTPS par rapport à HTTP).

1. Arrêt / désinstallation de TypSoft FTP Server.
2. Installation de **FileZilla Server** (supporte le FTPS), reprise de la configuration des comptes.
3. Génération d'un **certificat SSL auto-signé** directement dans FileZilla Server.
4. Activation du chiffrement TLS dans les paramètres du serveur.
5. Reconfiguration des clients : connexion en **FTPS explicite** (le certificat auto-signé doit être accepté manuellement à la première connexion).
6. **Nouvelle capture Wireshark** : le trafic est désormais chiffré et illisible → objectif de sécurité atteint.

Une sensibilisation des développeurs est réalisée (démonstration de la capture en clair puis chiffrée, reconfiguration du client en TLS).

Conclusion type

Ce TP a permis de mettre en place un service de transfert de fichiers fonctionnel (FTP), d'en démontrer concrètement les limites de sécurité (mot de passe en clair via Wireshark), puis de corriger le problème en migrant vers le **FTPS**. Il a aussi souligné l'importance de documenter séparément les usages développeur et technicien, et de sensibiliser les utilisateurs finaux aux bonnes pratiques.

À retenir pour l'examen :

- FTP transmet identifiants et données **en clair** (commandes USER/PASS) → FTPS = FTP + SSL/TLS pour chiffrer.
- Port FTP par défaut : **21**. Différence mode actif / passif (erreur 425 typique d'un souci de pare-feu en mode actif).
- Répartition des droits par profil (développeur / technicien / admin) — principe du moindre privilège.
- Deux publics = deux documentations différentes (utilisateur / technicien).
- Savoir citer Wireshark comme outil d'analyse réseau pour démontrer une faille.